

High Performance Computing

Introductory Task 1

The purpose of this task is to become familiar with the basic ideas of High Performance Computing (HPC) and to think about how a simple computation can be divided among several processing units.

No parallel programming libraries are required for this task.

Part 1: Basic HPC Concepts

Answer the following questions briefly.

1. What is High Performance Computing?
2. Give two examples of problems for which HPC may be useful.
3. Explain the difference between:
 - a processor/core,
 - a node,
 - a cluster.
4. Why are multiple processors useful for solving large computational problems?
5. Suppose a computer has 4 processors. Does this automatically mean that every program will run four times faster? Explain briefly.

Part 2: A Simple Computational Problem

Consider the following operation on an array:

$$a_i \leftarrow a_i + 1, \quad i = 0, 1, \dots, N - 1.$$

Write a C++ program that:

1. creates an array or vector containing $N = 1,000,000$ integers;
2. initializes
$$a_i = i;$$
3. increases every element by 1;
4. calculates the sum of all elements after the modification;
5. prints the final sum.

You may use the following program structure:

```

1 #include <iostream>
2 #include <vector>
3
4 int main() {
5
6     const int N = 1000000;
7
8     std::vector<long long> a(N);
9
10    // Initialize the array
11
12
13    // Increase every element by 1
14
15
16    // Calculate the sum
17
18
19    // Print the result
20
21
22    return 0;
23 }

```

Part 3: Thinking in Parallel

Assume that instead of one processor, we have **4 processors**:

$$P_0, \quad P_1, \quad P_2, \quad P_3.$$

The array contains $N = 1,000,000$ elements.

Divide the array into four approximately equal parts and complete the following table.

Processor	First index	Last index
P_0		
P_1		
P_2		
P_3		

Answer the following questions.

1. Can the four processors increase their parts of the array simultaneously?
2. Does processor P_0 need the result produced by P_1 before it can perform its calculations?
3. Which part of this computation can therefore be performed in parallel?
4. After all processors finish, what additional operation is needed if we want to calculate one final sum of the entire array?

Part 4: Small Experiment

Measure approximately how long the following loop takes on your computer:

```

1 for (int i = 0; i < N; ++i) {
2     a[i] = a[i] + 1;
3 }

```

You can use `std::chrono`:

```

1 #include <chrono>
2
3 auto start = std::chrono::high_resolution_clock::now();
4
5 // code to measure
6
7 auto finish = std::chrono::high_resolution_clock::now();
8
9 std::chrono::duration<double> elapsed = finish - start;
10
11 std::cout << "Time: "
12           << elapsed.count()
13           << " seconds\n";

```

Run the program for:

$$N = 10^5, \quad N = 10^6, \quad N = 10^7.$$

Record your results.

Array size N	Execution time (seconds)
10^5	
10^6	
10^7	

Part 5: Short Discussion

Answer in 2–4 sentences:

1. What happens to the execution time when N increases?
2. Why might using several processors be useful when N becomes very large?
3. What possible additional work may be required when several processors cooperate on the same problem?

Submission

Submit:

1. your C++ source file;
2. the completed tables;
3. answers to the short questions;
4. the measured execution times.